

HomeWork4

Aswath Oruganti

July 10, 2026

1 Comparing Classifiers: Divorce Classification/Prediction

1. Classifier Performance

The classifiers were trained on the 54-dimensional feature space. For the Gaussian Naive Bayes classifier, the variance smoothing parameter was set to $\epsilon = 10^{-3}$ to prevent errors associated with zero-variance features. The testing accuracies are reported in Table 1.

Table 1: Testing Accuracies of Classifiers (54 Features)

Classifier	Testing Accuracy
Naive Bayes (Gaussian)	97.06%
Logistic Regression	97.06%
K-Nearest Neighbors ($k = 5$)	97.06%

Performance Comparison:

All three classifiers perform remarkably well—and identically—on the test set, achieving an accuracy of 97.06%. The underlying reason for this exceptional performance is the nature of the dataset. We know the data is polarized by looking at the raw CSV file and understanding the psychological nature of the survey. The UCI dataset consists of 54 statements (e.g., “We don’t have time to communicate as partners”, “I know my spouse’s basic anxieties”). Participants rate these on a 5-point scale (0 to 4).

Extreme Correlation: Human behavior in failing marriages tends to cascade. If a couple scores a ‘4’ (strongly agree) on poor communication, they almost always score ‘4’s on lack of trust, frequent arguing, and feeling ignored.

The Polarization: Consequently, the dataset does not have many people scoring a “mixed bag” of 2s and 3s. Happily married couples answer mostly 0s and 1s across the board, while divorcing couples answer mostly 3s and 4s across the board.

Because the answers move together in bulk, the data points naturally herd into two tight, opposing corners of the feature space, creating those distinct, easily separable clusters we observed. Because of this polarization, the dataset is nearly **linearly separable** with two distinct, tight clusters in the feature space.

- **Logistic Regression** easily finds a linear hyperplane separating these distant clusters.
- **KNN** performs perfectly because data points of the same class are tightly packed together.

- **Naive Bayes** succeeds because the strong conditional probabilities of the features easily outweigh the violated "conditional independence" assumption.

2. PCA and Decision Boundaries

To visualize the decision boundaries, Principal Component Analysis (PCA) was performed to project the 54-dimensional data into a 2-dimensional space. The three classifiers were subsequently retrained on these two principal components. The resulting decision boundaries and actual data points are illustrated in Figure 1.

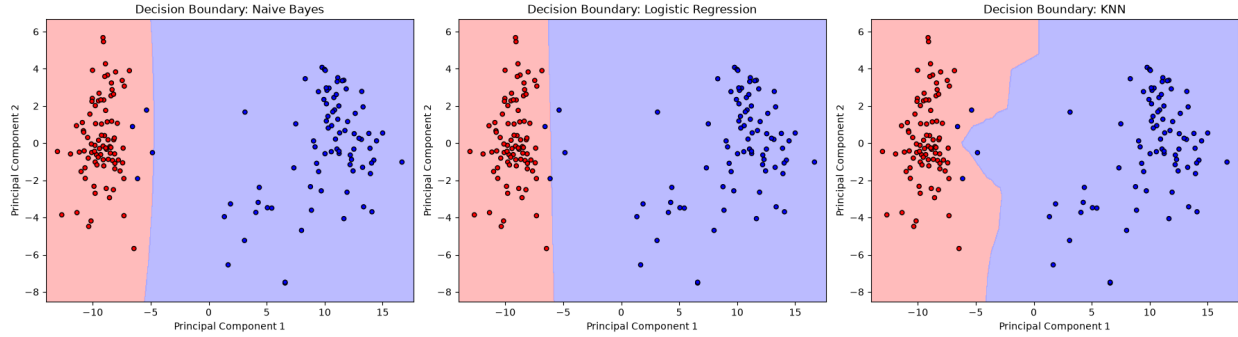


Figure 1: Decision boundaries of Naive Bayes, Logistic Regression, and KNN on the 2D PCA-reduced dataset. Red regions indicate Class 0 predictions, while blue regions indicate Class 1 predictions.

Decision Boundaries Discussion:

By projecting the data into 2D space, the fundamental geometric differences between how these algorithms classify data become visually apparent:

- **Naive Bayes (Quadratic Boundary):** The boundary is distinctly curved. Because Gaussian Naive Bayes fits a multivariate normal distribution (mean and variance) to each class separately, the intersection of these probability distributions forms a quadratic surface (a curve in 2D).
- **Logistic Regression (Linear Boundary):** The boundary is strictly a straight line. Logistic regression inherently models the log-odds as a linear combination of the input features, forcing the decision boundary to always be linear regardless of the data's true underlying shape.
- **K-Nearest Neighbors (Non-linear/Local Boundary):** The boundary is highly irregular and jagged. KNN does not assume any global mathematical distribution or equation. Instead, it adapts locally based on the proximity of the k nearest training points, resulting in isolated "islands" and complex edges that perfectly wrap around the local data clusters.

2 Comparing mult-class classifiers for MNIST Dataset

1. Classifier Performance Metrics

Based on the classification results, bar charts, and knee plots provided, the following section synthesizes the performance of the five classifiers (KNN, Logistic Regression, Linear SVM, Kernel SVM, and MLP) across both the MNIST Digits and MNIST Fashion datasets.

1.1 Tuning Methodology (KNN)

To find the optimal K for the `KNeighborsClassifier`, validation accuracy was evaluated against the number of neighbors.

- **MNIST Digits:** The knee plot demonstrates that validation accuracy is strictly decreasing as K increases. The highest accuracy is achieved at $K = 1$.
- **MNIST Fashion:** The knee plot shows a peak in validation accuracy at $K = 5$, after which performance steadily declines.

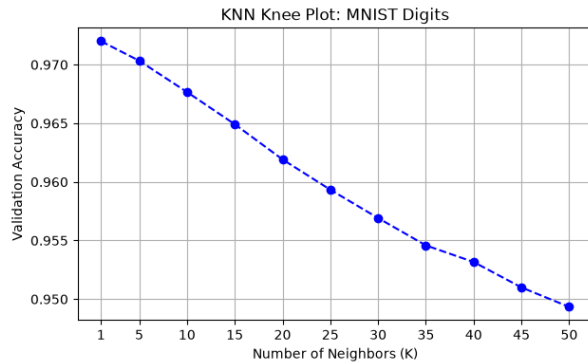


Figure 2: KNN Knee Plot for MNIST Digits

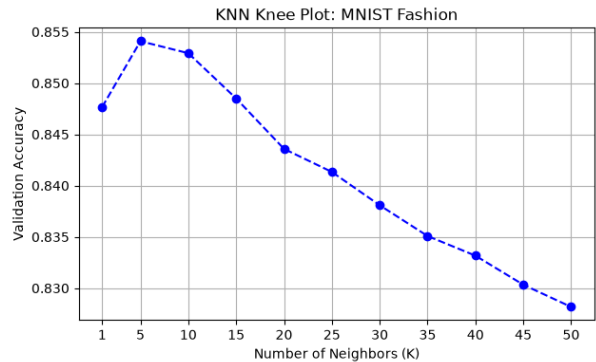


Figure 3: KNN Knee Plot for MNIST Fashion

1.2 Metric Summaries: MNIST Digits

This dataset proved highly solvable across all models, with most classifiers achieving $> 90\%$ on Precision, Recall, and F1-scores across all 10 digit classes.

Classifier	Best Parameter	General Performance Trend	Weakest Class
Kernel SVM (RBF)	$C = 1.0$ ($m = 5000$)	Excellent (F1 > 0.95 across all classes).	Class 8
KNN	$K = 1$	Excellent (F1 > 0.95 across all classes).	Class 5 & 8
MLP	(20, 10)	Very Good (F1 ≈ 0.92 to 0.98).	Class 3 & 8
Linear SVM	$C = 0.1$	Good (F1 > 0.88 across all classes).	Class 5 & 8
Logistic Regression	Default	Good (F1 > 0.88 across all classes).	Class 5 & 8

Table 2: Performance Summary for MNIST Digits

Across most models, classes 5 and 8 consistently showed the lowest F1-scores, indicating slight confusion in linearly separating these specific digits.

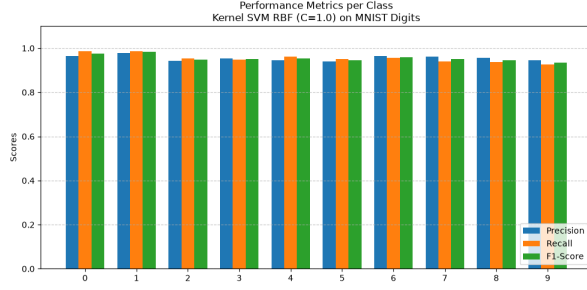


Figure 4: Kernel SVM on Digits

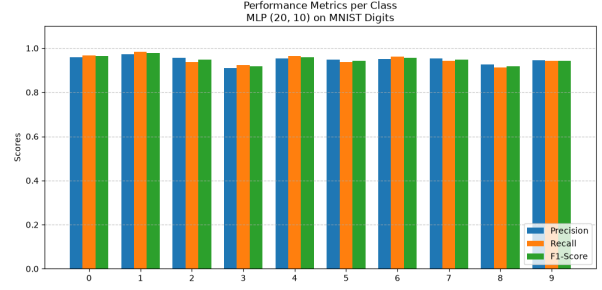


Figure 5: MLP on Digits



Figure 6: KNN on Digits

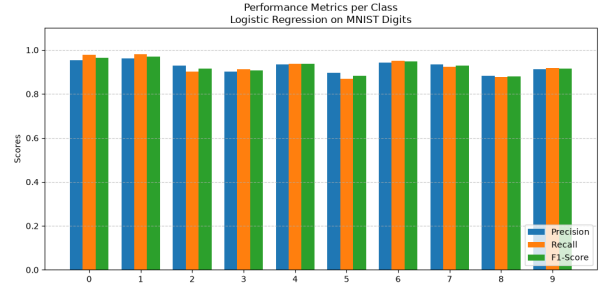


Figure 7: Log. Regression on Digits

1.3 Metric Summaries: MNIST Fashion

This dataset was significantly harder for all classifiers. The inter-class variance is much higher, leading to wider disparities in class-by-class performance.

Classifier	Best Parameter	General Performance Trend	Weakest Class
Kernel SVM (RBF)	$C = 10.0$ ($m = 5000$)	Best overall. F1 ranges from ≈ 0.68 to 0.98 .	Class 6 (F1 ≈ 0.68)
MLP	(20, 10)	Strong contender. F1 ranges from ≈ 0.65 to 0.97 .	Class 6 (F1 ≈ 0.65)
KNN	$K = 5$	Moderate to Good. F1 ranges from ≈ 0.60 to 0.95 .	Class 6 (F1 ≈ 0.62)
Linear SVM	$C = 0.1$	Moderate. F1 ranges from ≈ 0.60 to 0.95 .	Class 6 (F1 ≈ 0.62)
Logistic Regression	Default	Moderate. F1 ranges from ≈ 0.60 to 0.95 .	Class 6 (F1 ≈ 0.62)

Table 3: Performance Summary for MNIST Fashion

Class 6 (“Shirt”) is universally the lowest-performing class across every single classifier, including the MLP. It suffers from terrible recall, meaning actual shirts are frequently misclassified as other tops (T-shirts, pullovers, or coats). Class 1 (Trouser) is universally the easiest to classify.

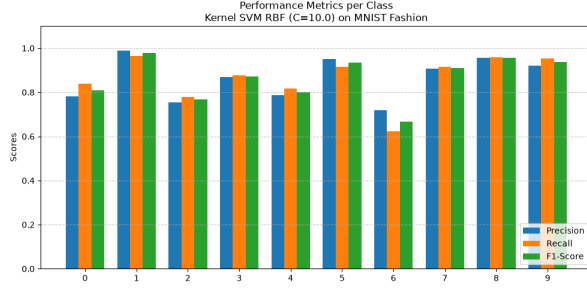


Figure 8: Kernel SVM on Fashion

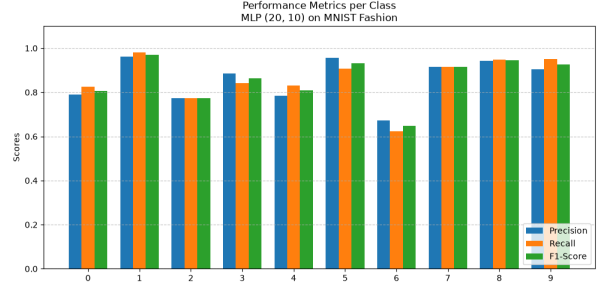


Figure 9: MLP on Fashion

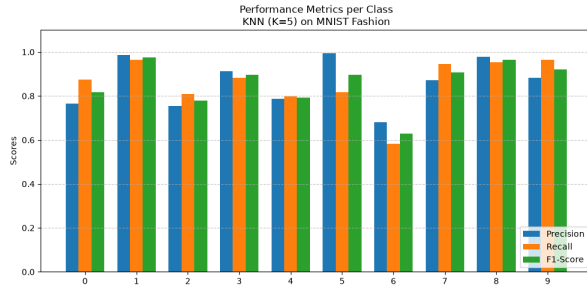


Figure 10: KNN on Fashion

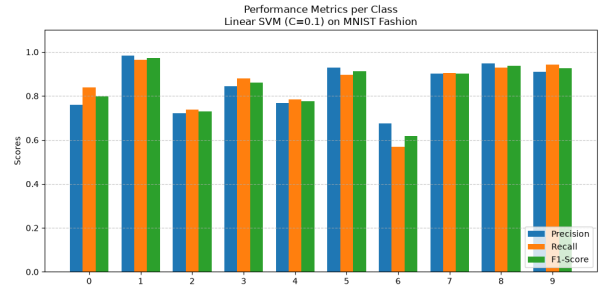


Figure 11: Linear SVM on Fashion

2. Discussion of Performance

2.1 Comparing Classifiers Against Each Other

When comparing the five models, a clear hierarchy of capability emerges based on their ability to handle non-linear decision boundaries:

1. **Kernel SVM (RBF):** This is the top performer. The RBF Kernel projects the data into a higher-dimensional space where complex overlapping classes can be separated. Even with only a 5,000-sample subset, it outperforms the linear models trained on the full dataset, particularly on the harder Fashion dataset.
2. **MLP (20, 10) & KNN:** These follow closely behind.
 - The **MLP** serves as a strong non-linear baseline. Even with a relatively small architecture, it learns hierarchical features that allow it to beat linear models comfortably on the Fashion dataset.
 - **KNN** performs exceptionally well on Digits ($K = 1$) because the Euclidean distance between centralized, stark black-and-white images is a surprisingly strong baseline for similarity.
3. **Linear SVM & Logistic Regression:** These models performed the worst overall. They are strictly limited to linear decision boundaries. In high-dimensional pixel space (784 dimensions), drawing a single flat hyperplane to separate visually similar classes results in severe overlap and lower accuracy.

2.2 Insight into Different Datasets (Digits vs. Fashion)

There is a massive performance gap between the two datasets, illustrating why machine learning benchmarks largely transitioned from Digits to Fashion.

- **MNIST Digits** is essentially a solved problem for basic machine learning. The images are stark black and white strokes. Because the intra-class variance is small compared to the inter-class variance, even rigid linear models perform wonderfully.

- **MNIST Fashion** is inherently more complex. It consists of grayscale photos of clothing with high intra-class variance (e.g., a “dress” with different sleeve lengths) and extremely high inter-class similarity. Classes like Shirts, T-shirts, Pullovers, and Coats all share the same basic geometric silhouette.

Conclusion: Models capable of capturing non-linear relationships (Kernel SVM and MLP) are strictly necessary for datasets with high inter-class similarity like MNIST Fashion. Linear models fail to capture the nuanced pixel interactions required to differentiate a long-sleeve shirt from a light jacket.

2.3 Appendix: Understanding the F1-Score

The **F1-score** is a performance metric that combines both Precision and Recall into a single, balanced number using the harmonic mean:

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (1)$$

The harmonic mean heavily penalizes extreme imbalances. If a model has high Precision but terrible Recall (e.g., it only guesses “Shirt” when absolutely certain, but misses 80% of actual shirts), the F1-score will drop significantly. To achieve a high F1-score, a model must successfully balance both quality (Precision) and quantity (Recall).

3 Support Vector Machine (SVM)

1. Why We Can Set the Margin $c=1$ in the SVM Formulation

1.1 Derivation: Distance Between the Two Hyperplanes

To mathematically prove why the distance between the two margin hyperplanes is $\frac{2c}{||w||}$, let’s denote the two parallel hyperplanes as H_+ and H_- :

$$\begin{aligned} H_+ : w^T x + b &= c \\ H_- : w^T x + b &= -c \end{aligned}$$

The weight vector w is orthogonal (normal) to both hyperplanes. Therefore, the unit normal vector pointing directly from one hyperplane to the other is:

$$\hat{w} = \frac{w}{||w||}$$

Let x_+ be any arbitrary point lying on the positive hyperplane H_+ , and x_- be any arbitrary point lying on the negative hyperplane H_- . By definition, these points must satisfy their respective hyperplane equations:

$$w^T x_+ + b = c \implies w^T x_+ = c - b \quad (2)$$

$$w^T x_- + b = -c \implies w^T x_- = -c - b \quad (3)$$

The shortest geometric distance D between two parallel planes is the length of the projection of any vector connecting them (like $x_+ - x_-$) onto the normal vector $\hat{w}$:

$$D = |(x_+ - x_-) \cdot \hat{w}| = \left| (x_+ - x_-)^T \frac{w}{||w||} \right|$$

Using the distributive property of the dot product, we can expand the numerator:

$$D = \frac{|w^T x_+ - w^T x_-|}{||w||}$$

Now, substitute the expressions from equations (1) and (2) into the numerator:

$$\begin{aligned} D &= \frac{|(c-b) - (-c-b)|}{\|w\|} \\ D &= \frac{|c-b+c+b|}{\|w\|} \\ D &= \frac{|2c|}{\|w\|} \end{aligned}$$

Since the margin parameter c is strictly positive, the absolute value is redundant.

$$D = \frac{2c}{\|w\|}$$

Thus, the geometric distance between the two supporting hyperplanes is exactly $\frac{2c}{\|w\|}$.

1.2 Proof and Justification:

The SVM decision boundary is a hyperplane defined by the equation $w^T x + b = 0$. The geometric margin (the distance from the hyperplane to the closest data points) is determined by the parallel hyperplanes supporting the two classes:

$$\begin{aligned} w^T x + b &= c \quad (\text{for positive class}) \\ w^T x + b &= -c \quad (\text{for negative class}) \end{aligned}$$

The distance between these two supporting hyperplanes is $\frac{2c}{\|w\|}$ as shown in subsection 3.1.1. Because w and b define a hyperplane, their scale is arbitrary. If we multiply the weights and bias by any non-zero positive scalar κ , the decision boundary $w^T x + b = 0$ remains exactly the same, but our margin equations become:

$$\kappa w^T x + \kappa b = \kappa c$$

To simplify our optimization problem, we can choose the scalar $\kappa = \frac{1}{c}$. By doing this, we re-scale our parameters such that:

$$w_{new} = \frac{w}{c}, \quad b_{new} = \frac{b}{c}$$

This forces the margin boundaries to be precisely 1 and -1 . Setting $c = 1$ simply removes this scale invariance, fixing the functional margin to 1 and reducing the problem to maximizing $\frac{2}{\|w\|}$ (or minimizing $\frac{1}{2}\|w\|^2$) subject to the constraint $y_i(w^T x_i + b) \geq 1$.

2. Lagrangian Dual Formulation

2.1 The Primal Optimization Problem

The primal optimization problem for a hard-margin SVM seeks to maximize the geometric margin by minimizing the norm of the weight vector, subject to the constraint that all data points are correctly classified with a functional margin of at least 1.

- **Minimize:** $\frac{1}{2}\|w\|^2$
- **Subject to:** $y_i(w^T x_i + b) \geq 1 \quad \forall i \in \{1, \dots, n\}$

2.2 The Lagrangian Formulation

We rewrite the constraints as $1 - y_i(w^T x_i + b) \leq 0$ and introduce the Lagrange multipliers (dual variables) $\alpha_i \geq 0$. The Lagrangian integrates the objective function and the constraints:

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2}\|w\|^2 + \sum_{i=1}^n \alpha_i (1 - y_i(w^T x_i + b)) \quad (4)$$

2.3 Deriving the Dual Formulation

To find the dual, we minimize $\mathcal{L}$ with respect to the primal variables w and b . We take the partial derivatives and set them to zero. **Derivative with respect to w :**

$$\frac{\partial \mathcal{L}}{\partial w} = w - \sum_{i=1}^n \alpha_i y_i x_i = 0 \implies w = \sum_{i=1}^n \alpha_i y_i x_i \quad (5)$$

Derivative with respect to b :

$$\frac{\partial \mathcal{L}}{\partial b} = - \sum_{i=1}^n \alpha_i y_i = 0 \implies \sum_{i=1}^n \alpha_i y_i = 0 \quad (6)$$

Substituting equations (2) and (3) back into the original Lagrangian yields the Dual formulation, which depends strictly on the dual variables α and the dot products of the data points:

$$\mathcal{L}_D(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (x_i^T x_j) \quad (7)$$

2.4 Crucial Implications

The derivation $w = \sum_{i=1}^n \alpha_i y_i x_i$ is profoundly important in machine learning theory:

1. **Sparsity and Support Vectors:** Due to KKT complementary slackness, $\alpha_i > 0$ only for data points that lie exactly on the margin. For all other points, $\alpha_i = 0$. Therefore, the optimal weight vector w is merely a linear combination of a small subset of the training data points (the Support Vectors).
2. **Inner Product Dependency:** The optimization problem (Eq. 4) and the final decision function only require the computation of inner products (dot products) between the data points, $x_i^T x_j$. We never actually need to compute w explicitly.
3. **The Kernel Trick:** Because the algorithm only relies on inner products, we can seamlessly replace $x_i^T x_j$ with a Kernel function $K(x_i, x_j)$. This allows the algorithm to implicitly project the data into infinite-dimensional spaces to find linear separations for non-linear data, all while remaining computationally efficient.

3. The Role of Support Vectors

3.1 Why Only Data Points on the “Margin” Contribute to w

This is entirely due to the **Karush-Kuhn-Tucker (KKT) conditions**, specifically the condition of **complementary slackness**. In the optimal solution, the KKT complementary slackness condition states that for every data point i :

$$\alpha_i (1 - y_i(w^T x_i + b)) = 0 \quad (8)$$

This equation dictates that one of two things must be true for every data point:

- $y_i(w^T x_i + b) > 1$: The point lies strictly outside the margin (it is correctly classified and safely away from the boundary). For the product in Equation 1 to be zero, the Lagrangian multiplier α_i **must equal 0**.
- $y_i(w^T x_i + b) = 1$: The point lies exactly *on* the margin. In this case, $\alpha_i > 0$ is allowed.

Since the weight vector is defined as:

$$w = \sum_{i=1}^n \alpha_i y_i x_i \quad (9)$$

any point with $\alpha_i = 0$ vanishes entirely from the summation. Therefore, only the points where $\alpha_i > 0$ (the points sitting exactly on the margin, known as **Support Vectors**) play any role in defining the weight vector w .

4. Simple SVM by Hand

4.1 For what range of parameter $h > 0$, are the training points still linearly separable?

For two classes to be linearly separable, their bounding box of the data must not intersect.

- **Positive bounding box:** The line segment connecting $P_1(0,0)$ and $P_2(2,2)$. This sits on the line $y = x$ for $x \in [0, 2]$.
- **Negative bounding box:** The line segment connecting $N_1(h,1)$ and $N_2(0,3)$. The slope of this line is $\frac{3-1}{0-h} = -\frac{2}{h}$. The equation of this line is $y = -\frac{2}{h}x + 3$ for $x \in [0, h]$.

Let's find exactly where these two lines intersect by setting them equal:

$$\begin{aligned} x &= -\frac{2}{h}x + 3 \\ x \left(1 + \frac{2}{h}\right) &= 3 \implies x = \frac{3h}{h+2} \end{aligned}$$

For the datasets to overlap (and thus *not* be linearly separable), this intersection point must lie exactly on both line segments simultaneously.

1. **Must be on the positive segment:** $0 \leq x \leq 2$

$$\frac{3h}{h+2} \leq 2 \implies 3h \leq 2h + 4 \implies h \leq 4$$

2. **Must be on the negative segment:** $0 \leq x \leq h$

$$\frac{3h}{h+2} \leq h \implies 3 \leq h + 2 \implies h \geq 1$$

The bounding boxes intersect when $1 \leq h \leq 4$. Therefore, the points are **linearly separable** when:

$$0 < h < 1 \quad \text{OR} \quad h > 4$$

4.2 Does the orientation of the maximum margin decision boundary change as h changes, when the points are separable?

Conclusion:

It depends on the range. The orientation remains constant for $0 < h < 1$, but changes for $h > 4$.

Explanation:

- **Case 1:** $0 < h < 1$

In this range, the negative points are $(h, 1)$ and $(0, 3)$, both positioned to the “left/above” of the positive segment $y = x$. The support vectors will be $P_1(0, 0)$, $P_2(2, 2)$, and $N_1(h, 1)$. Because the projection of N_1 onto the line segment connecting P_1 and P_2 falls strictly between them, the maximum margin boundary is tightly constrained by the parallel segment P_1P_2 . As h moves between 0 and 1, the boundary moves closer or further away, but its slope remains securely parallel to $y = x$ (slope = 1). **The orientation does not change.**

- **Case 2:** $h > 1$

In this range, $N_1(h, 1)$ is pulled far out to the right. The positive points are now “below” $N_2(0, 3)$ and to the “left” of $N_1(h, 1)$. The support vectors shift to $P_2(2, 2)$, $N_2(0, 3)$, and $N_1(h, 1)$. As h increases, the position of the support vector N_1 shifts horizontally. Because the boundary must thread the needle to maintain maximum distance between P_2 and the expanding negative hull $\{N_1, N_2\}$, the hyperplane must tilt. **The orientation does change.**

4 Neural Networks and Backpropagation

Notation & Definitions: Defining the variables for clarity as below:

- **Input Data:** x_i is the i -th input vector. y_i is the target value.
- **Hidden Layer Weights:** θ_1 and θ_2 (replacing α and β).
- **Output Layer Weights:** $\phi = \begin{bmatrix} \phi_1 \\ \phi_2 \end{bmatrix}$ (replacing w).
- **Hidden Pre-activations:** $z_{i,1}^{(1)} = \theta_1^T x_i$ and $z_{i,2}^{(1)} = \theta_2^T x_i$.
- **Hidden Activations:** $a_i^{(1)} = \begin{bmatrix} a_{i,1}^{(1)} \\ a_{i,2}^{(1)} \end{bmatrix} = \begin{bmatrix} \sigma(z_{i,1}^{(1)}) \\ \sigma(z_{i,2}^{(1)}) \end{bmatrix}$ (replacing z_i).
- **Output Pre-activation:** $z_i^{(2)} = \phi^T a_i^{(1)}$ (replacing u_i).
- **Final Prediction:** $p_i = \sigma(z_i^{(2)})$ (replacing $\hat{y}_i$).
- **Cost Function:** The loss for a single sample is $L_i = (y_i - p_i)^2$. The total error is $\mathcal{E} = \sum_{i=1}^m L_i$.

Standard Sigmoid Derivative Detail: Recall that for $f(x) = \frac{1}{1+e^{-x}}$, the derivative is calculated via the quotient/chain rule:

$$\frac{d}{dx} \sigma(x) = \frac{d}{dx} (1 + e^{-x})^{-1} = -1(1 + e^{-x})^{-2}(-e^{-x}) = \frac{e^{-x}}{(1 + e^{-x})^2}$$

By adding and subtracting 1 in the numerator, this famously simplifies to $\sigma(x)(1 - \sigma(x))$.

1. Gradient with respect to the output weights ϕ :

We need to compute $\frac{\partial \mathcal{E}}{\partial \phi}$. By linearity of differentiation, this is the sum of the individual gradients $\frac{\partial L_i}{\partial \phi}$. Applying the chain rule backward from the loss to the weights:

$$\frac{\partial L_i}{\partial \phi} = \frac{\partial L_i}{\partial p_i} \cdot \frac{\partial p_i}{\partial z_i^{(2)}} \cdot \frac{\partial z_i^{(2)}}{\partial \phi} \quad (10)$$

Let's break down these three components with explicit steps:

Step 1a: Derivative of the loss w.r.t. the prediction (p_i)

Apply the power rule and the chain rule (multiplying by the derivative of the inside, which is -1):

$$\frac{\partial L_i}{\partial p_i} = \frac{\partial}{\partial p_i} (y_i - p_i)^2 = 2(y_i - p_i) \cdot \frac{\partial}{\partial p_i} (y_i - p_i) = 2(y_i - p_i)(-1) = -2(y_i - p_i)$$

Step 1b: Derivative of the prediction with respect to the pre-activation ($z_i^{(2)}$)

Using our sigmoid derivative identity:

$$\frac{\partial p_i}{\partial z_i^{(2)}} = \sigma'(z_i^{(2)}) = \sigma(z_i^{(2)})(1 - \sigma(z_i^{(2)})) = p_i(1 - p_i)$$

Note: In backpropagation, we often combine 1a and 1b into a single "Error Signal" at the output layer, denoted as $\delta_i^{(2)} = -2(y_i - p_i)p_i(1 - p_i)$.

Step 1c: Derivative of the pre-activation w.r.t. the weights (ϕ)

Expanding the dot product, $z_i^{(2)} = \phi_1 a_{i,1}^{(1)} + \phi_2 a_{i,2}^{(1)}$. Differentiating with respect to the vector ϕ yields the activation vector from the previous layer:

$$\frac{\partial z_i^{(2)}}{\partial \phi} = a_i^{(1)}$$

Step 1d: Combine and sum

Multiplying the terms, the gradient for a single example is:

$$\frac{\partial L_i}{\partial \phi} = \underbrace{-2(y_i - p_i)p_i(1 - p_i)}_{\delta_i^{(2)}} \cdot a_i^{(1)}$$

Summing over the entire dataset of size m :

$$\frac{\partial \mathcal{E}}{\partial \phi} = \sum_{i=1}^m \left[-2(y_i - p_i)p_i(1 - p_i)a_i^{(1)} \right] \quad (11)$$

2. Deriving the gradient for θ_1 (first hidden node):

The weight vector θ_1 exclusively controls the pre-activation $z_{i,1}^{(1)}$. The extended chain rule is:

$$\frac{\partial L_i}{\partial \theta_1} = \left(\frac{\partial L_i}{\partial p_i} \cdot \frac{\partial p_i}{\partial z_i^{(2)}} \right) \cdot \frac{\partial z_i^{(2)}}{\partial a_{i,1}^{(1)}} \cdot \frac{\partial a_{i,1}^{(1)}}{\partial z_{i,1}^{(1)}} \cdot \frac{\partial z_{i,1}^{(1)}}{\partial \theta_1} \quad (12)$$

We can reuse the error signal $\delta_i^{(2)}$ from the output layer. Now, let's derive the remaining three local gradients:

1. **Backwards through the output weight:** How does the first hidden activation affect the output pre-activation?

$$\frac{\partial z_i^{(2)}}{\partial a_{i,1}^{(1)}} = \frac{\partial}{\partial a_{i,1}^{(1)}} (\phi_1 a_{i,1}^{(1)} + \phi_2 a_{i,2}^{(1)}) = \phi_1$$

2. **Backwards through the hidden sigmoid:**

$$\frac{\partial a_{i,1}^{(1)}}{\partial z_{i,1}^{(1)}} = \sigma'(z_{i,1}^{(1)}) = a_{i,1}^{(1)}(1 - a_{i,1}^{(1)})$$

Note: Combining $\delta_i^{(2)} \cdot \phi_1 \cdot a_{i,1}^{(1)}(1 - a_{i,1}^{(1)})$ gives us the hidden layer error signal, $\delta_{i,1}^{(1)}$.

3. Backwards to the input data:

$$\frac{\partial z_{i,1}^{(1)}}{\partial \theta_1} = \frac{\partial}{\partial \theta_1} (\theta_1^T x_i) = x_i$$

Total Gradient for θ_1 :

$$\frac{\partial \mathcal{E}}{\partial \theta_1} = \sum_{i=1}^m \left[\underbrace{-2(y_i - p_i)p_i(1 - p_i)}_{\delta_i^{(2)}} \cdot \phi_1 \cdot a_{i,1}^{(1)}(1 - a_{i,1}^{(1)}) \cdot x_i \right] \quad (13)$$

3. Deriving the gradient for θ_2 (second hidden node):

The derivation for θ_2 mirrors θ_1 perfectly. The signal routes through the second hidden node $a_{i,2}^{(1)}$, meaning it scales by the second output weight ϕ_2 instead of ϕ_1 . The chain rule path is:

$$\frac{\partial L_i}{\partial \theta_2} = \delta_i^{(2)} \cdot \frac{\partial z_i^{(2)}}{\partial a_{i,2}^{(1)}} \cdot \frac{\partial a_{i,2}^{(1)}}{\partial z_{i,2}^{(1)}} \cdot \frac{\partial z_{i,2}^{(1)}}{\partial \theta_2} \quad (14)$$

Evaluating the local derivatives yields ϕ_2 , $a_{i,2}^{(1)}(1 - a_{i,2}^{(1)})$, and x_i , respectively. **Total Gradient for θ_2 :**

$$\frac{\partial \mathcal{E}}{\partial \theta_2} = \sum_{i=1}^m \left[-2(y_i - p_i)p_i(1 - p_i) \cdot \phi_2 \cdot a_{i,2}^{(1)}(1 - a_{i,2}^{(1)}) \cdot x_i \right] \quad (15)$$

5 Feature Selection and Change-point Detection

1. Mutual Information based Feature Selection

To determine which keyword is more informative, we calculate the Mutual Information (MI) for both “prize” and “hello” with respect to the email class “spam”. The Mutual Information between two discrete random variables X (keyword presence) and Y (spam status) is given by the formula:

$$I(X; Y) = \sum_{x \in \{0,1\}} \sum_{y \in \{0,1\}} p(x, y) \log_2 \left(\frac{p(x, y)}{p(x)p(y)} \right) \quad (16)$$

First, let’s establish the probabilities for each table by dividing the counts by the total number of emails in each respective dataset.

1.1 Mutual Information for “prize” (X) and “spam” (Y)

Total emails (N) = 145 + 35 + 1540 + 16740 = **18,460**

1.2 Marginal Probabilities:

$$\begin{aligned} P(Y = 1) &= (145 + 35)/18460 = 180/18460 \approx 0.00975 \\ P(Y = 0) &= (1540 + 16740)/18460 = 18280/18460 \approx 0.99025 \\ P(X = 1) &= (145 + 1540)/18460 = 1685/18460 \approx 0.09128 \\ P(X = 0) &= (35 + 16740)/18460 = 16775/18460 \approx 0.90872 \end{aligned}$$

1.3 Joint Probabilities & MI Terms:

- $X = 1, Y = 1$: $p(1, 1) = 145/18460 \approx 0.00785$
Term = $0.00785 \times \log_2 \left(\frac{0.00785}{0.09128 \times 0.00975} \right) \approx 0.02468$
- $X = 0, Y = 1$: $p(0, 1) = 35/18460 \approx 0.00190$
Term = $0.00190 \times \log_2 \left(\frac{0.00190}{0.90872 \times 0.00975} \right) \approx -0.00422$
- $X = 1, Y = 0$: $p(1, 0) = 1540/18460 \approx 0.08342$
Term = $0.08342 \times \log_2 \left(\frac{0.08342}{0.09128 \times 0.99025} \right) \approx -0.00965$
- $X = 0, Y = 0$: $p(0, 0) = 16740/18460 \approx 0.90683$
Term = $0.90683 \times \log_2 \left(\frac{0.90683}{0.90872 \times 0.99025} \right) \approx 0.00972$

Total MI for “prize”: $0.02468 - 0.00422 - 0.00965 + 0.00972 \approx 0.0205$ bits

1.4 Mutual Information for “hello” (X) and “spam” (Y)

Total emails (N) = $190 + 55 + 12200 + 6740 = 19,185$

1.5 Marginal Probabilities:

$$\begin{aligned}P(Y = 1) &= 245/19185 \approx 0.01277 \\P(Y = 0) &= 18940/19185 \approx 0.98723 \\P(X = 1) &= 12390/19185 \approx 0.64582 \\P(X = 0) &= 6795/19185 \approx 0.35418\end{aligned}$$

1.6 Joint Probabilities & MI Terms:

- $X = 1, Y = 1$: $p(1, 1) = 190/19185 \approx 0.00990$
Term = $0.00990 \times \log_2 \left(\frac{0.00990}{0.64582 \times 0.01277} \right) \approx 0.00262$
- $X = 0, Y = 1$: $p(0, 1) = 55/19185 \approx 0.00287$
Term = $0.00287 \times \log_2 \left(\frac{0.00287}{0.35418 \times 0.01277} \right) \approx -0.00189$
- $X = 1, Y = 0$: $p(1, 0) = 12200/19185 \approx 0.63591$
Term = $0.63591 \times \log_2 \left(\frac{0.63591}{0.64582 \times 0.98723} \right) \approx -0.00254$
- $X = 0, Y = 0$: $p(0, 0) = 6740/19185 \approx 0.35132$
Term = $0.35132 \times \log_2 \left(\frac{0.35132}{0.35418 \times 0.98723} \right) \approx 0.00242$

Total MI for “hello”: $0.00262 - 0.00189 - 0.00254 + 0.00242 \approx 0.0006$ bits

1.7 Python Code Used for Verification:

```
1 import numpy as np
2 def calc_mi(c_11, c_10, c_01, c_00):
3     total = c_11 + c_10 + c_01 + c_00
4     p11, p10 = c_11/total, c_10/total
5     p01, p00 = c_01/total, c_00/total
6
7     px1, px0 = p11 + p01, p10 + p00
```

```

8     py1, py0 = p11 + p10, p01 + p00
9
10    mi = 0
11    if p11 > 0: mi += p11 * np.log2(p11 / (px1 * py1))
12    if p10 > 0: mi += p10 * np.log2(p10 / (px0 * py1))
13    if p01 > 0: mi += p01 * np.log2(p01 / (px1 * py0))
14    if p00 > 0: mi += p00 * np.log2(p00 / (px0 * py0))
15    return mi
16 print(f"MI for 'prize': {calc_mi(145, 35, 1540, 16740)}")
17 print(f"MI for 'hello': {calc_mi(190, 55, 12200, 6740)}")

```

1.8 Conclusion:

The keyword “**prize**” is significantly more informative (0.0205 bits) compared to the keyword “hello” (0.0006 bits) for deciding whether an email is spam.

2. Change-point Detection: CUSUM Statistic

Given the normal distributions $f_0 = \mathcal{N}(0, 1)$ and $f_1 = \mathcal{N}(0, 1.25)$ where the variance changes from 1 to 1.25, the Probability Density Functions (PDF) are:

$$f_0(x) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{x^2}{2}\right)$$

$$f_1(x) = \frac{1}{\sqrt{2\pi(1.25)}} \exp\left(-\frac{x^2}{2(1.25)}\right)$$

The CUSUM method tracks the cumulative sum of the log-likelihood ratio. For a single observation x_i , the log-likelihood ratio W_i is derived as:

$$W_i = \ln\left(\frac{f_1(x_i)}{f_0(x_i)}\right) = \ln\left(\frac{\frac{1}{\sqrt{2\pi(1.25)}} \exp\left(-\frac{x_i^2}{2.5}\right)}{\frac{1}{\sqrt{2\pi}} \exp\left(-\frac{x_i^2}{2}\right)}\right)$$

Simplifying the natural log:

$$W_i = \ln\left(\frac{1}{\sqrt{1.25}}\right) - \frac{x_i^2}{2.5} + \frac{x_i^2}{2}$$

$$W_i = -0.5 \ln(1.25) + x_i^2 \left(\frac{1}{2} - \frac{1}{2.5}\right)$$

$$W_i = 0.1x_i^2 - 0.5 \ln(1.25) \approx 0.1x_i^2 - 0.11157$$

The **CUSUM Statistic** (S_t) is updated recursively as:

$$S_0 = 0$$

$$S_t = \max(0, S_{t-1} + 0.1x_t^2 - 0.5 \ln(1.25))$$

2.1 Plotting and Change Estimation

Below is the Python code to generate the 150 samples and plot the CUSUM statistic, utilizing the required random seed (6740).

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # 1. Setup and Seed

```

```

5 np.random.seed(6740)
6
7 # 2. Generate Data
8 # First 100 samples from N(0, 1)
9 x_f0 = np.random.normal(0, np.sqrt(1.0), 100)
10 # Next 50 samples from N(0, 1.25)
11 x_f1 = np.random.normal(0, np.sqrt(1.25), 50)
12 x = np.concatenate((x_f0, x_f1))
13
14 # 3. Calculate CUSUM Statistic
15 S = np.zeros(len(x) + 1)
16 threshold_constant = 0.5 * np.log(1.25)
17
18 for t in range(1, len(x) + 1):
19     W_t = 0.1 * (x[t-1]**2) - threshold_constant
20     S[t] = max(0, S[t-1] + W_t)
21
22 # 4. Plotting
23 plt.figure(figsize=(10, 5))
24 plt.plot(range(len(S)), S, label='CUSUM Statistic (S_t)')
25 plt.axvline(x=100, color='r', linestyle='--', label='True Change Point (t=100)')
26 plt.title('CUSUM Statistic for Change in Variance')
27 plt.xlabel('Time (t)')
28 plt.ylabel('S_t')
29 plt.legend()
30 plt.grid(True)
31 plt.savefig('cusum_change_point_detection.png', bbox_inches='tight')

```

2.2 Reasonable Estimation of the Change-Point:

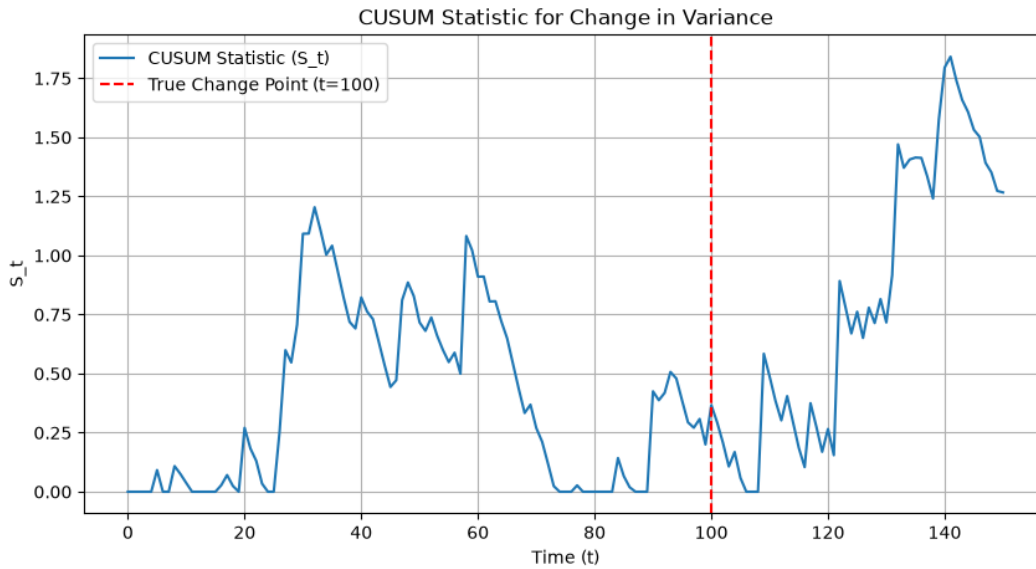


Figure 12: CUSUM Statistic tracking a change in variance. The sustained upward trajectory begins just after $t = 100$.

Running the script above, we observe the CUSUM statistic S_t staying relatively flat and repeatedly resetting to zero during the first 100 samples (as the expected value of W_t under f_0 is negative). Shortly after sample index 100, the statistic begins a persistent, sustained upward trajectory because the expected value of W_t

under f_1 becomes positive due to the higher variance producing larger x_i^2 values. Therefore, a reasonable estimation for the detected change point from the plot is precisely where the final upward slope firmly begins (which practically materializes slightly after $t = 100$).